

Softwaregestützte Modellbildung und Simulation: Exercise 2

Principal Component Analysis (PCA)

Peleg Bar Sapir

June 5, 2026

Principal Component Analysis (PCA) is an application of Singular Value Decomposition (SVD) to a data set, which allows us to visualize the data by *dimensionality reduction*: instead of looking at a high-dimensional data (sometimes in excess of several *hundreds* dimensions!), which may have numerous correlations - we can look at projections of said data on the first few directions in which its variance is the largest.

In standard PCA, the data set is represented as a matrix, with its **rows** being the separate *samples* (measurements), and its **columns** the different *variables* (features).

The following steps describe how to implement PCA in Python using NumPy's `SVD()` function, and visualize the resulting data representation using Matplotlib's `axes.bars()`, `axes.plot()` and `axes.scatter()` functions.

1. Load the data into a single NumPy array, using `numpy.loadtxt()`. Mathematically, we will call this data array **X**. *sepal length*, *sepal width*, *petal length* and *petal width*.
2. Center and normalize the data by subtracting from each **column** of **x** its mean, then dividing it by its standard deviation. Use the functions `numpy.mean()` and `numpy.std()`, respectively. Which axis should you use for these operations? Call the resulting matrix **B**.
3. Apply the SVD method to the matrix **B**:

```
1 U, S, VT = numpy.linalg.SVD(B, full_matrices=False)
```

(where **U**, **S**, **VT** are, respectively, the matrices **U**, **Σ** , and **V^T** in the SVD of **B**)

4. In the terms of PCA, the columns of $\mathbf{U}$ are called the *Principal Components* (PC) of $\mathbf{B}$. The number of principal components N is the number of variables in the data:

$$\text{PC}_1, \text{PC}_2, \dots, \text{PC}_N.$$

The corresponding Singular Values are the diagonal elements of the matrix $\mathbf{\Sigma}$, denoted as

$$\sigma_1, \sigma_2, \dots, \sigma_N.$$

The singular values are ordered by absolute value in descending order. The variance in PC_i is given by

$$\text{Var}_i = \frac{\sigma_i^2}{r - 1},$$

where r is the number of *samples* and $i = 1, 2, \dots, N$.

Plot the variance for each PC in descending order.

5. What do we get if we divide each Var_i by the sum of all variance terms? Plot these values as function of i . (**hint**: what is the sum of these terms?)
6. Plot the *cumulative sum* of the values calculated in the previous step, again against i (use the function `numpy.cumsum()`). Does the cumulative sum seem to converge, and if so - to what limit? Explain briefly.
7. Since the PCs are ordered by their variance in descending order, we can choose the first two PCs, PC_1 and PC_2 , and project the data on to them. This will keep the most variance in the data while allowing us to visualize it. The matrix $\mathbf{U}$ has the PCs as its columns, and $\mathbf{\Sigma}$ are their respective values - both are sorted by variance in descending order. Therefore, the full data in terms of the PCs is given by

$$\mathbf{P} = \mathbf{U} \cdot \mathbf{\Sigma}. \quad (1)$$

The projection of the data on the first k PCs is then given by the matrix product of the first k rows of $\mathbf{U}$ and the first k rows of the matrix $\mathbf{\Sigma}$:

$$\mathbf{P}_{j=1}^k = \mathbf{U}_{j=1}^k \cdot \mathbf{\Sigma}_{j=1}^k. \quad (2)$$

Implement this last projection (Equation 2) on the first two PCs (i.e. $k = 2$), and plot the result as a scatter plot of PC_2 vs. PC_1 . Use `axes.scatter(x, y)`, where $\mathbf{x}$ and $\mathbf{y}$ are the components of the data points in the PC_1 and PC_2 basis vectors, respectively.

8. You can color the data using the last column, which separates the samples according to the plant's type: Setosa, Versicolor and Virginica. These are represented in the data set as 0, 1 and 2, respectively. To use these columns as the color in the scatter plot from the previous step, use the following steps:

a) Import the following two functions from Matplotlib's `colors` module:

```
1 from matplotlib.colors import ListedColormap, BoundaryNorm
```

b) After loading the data into a matrix, save the last column as the plant's type (make sure this array contains only the values 0, 1 and 2).

```
1 species = data[:, -1]
```

c) Generate a color map where 0 is mapped to `red`, 1 to `green` and 2 to `blue` (or whatever other three colors you want):

```
1 colors = ["red", "green", "blue"]
2 cmap = ListedColormap(colors)
3 bounds = [-0.1, 0.9, 1.9, 2.9]
4 norm = BoundaryNorm(bounds, cmap.N)
```

d) Apply the color map to your scatter plot (note that the first two arguments are hidden since they relate to the task in the previous step):

```
1 ax.scatter(..., ..., c=species, cmap=cmap, norm=norm)
```